MANAJEMEN BASIS DATA

Index dan Security

Dosen Pengampu:

Satriyo Adhy, S.Si., M.T

DISUSUN OLEH

Nayla Husna	24060124140158
Khanza Qaila	24060124120011
Muhammad Akmal Fazli Riyadi	24060124130123
Muhammad Fahri	24060124120037
Muhammad Fikri	24060124130069

PROGRAM STUDI S1-INFORMATIKA

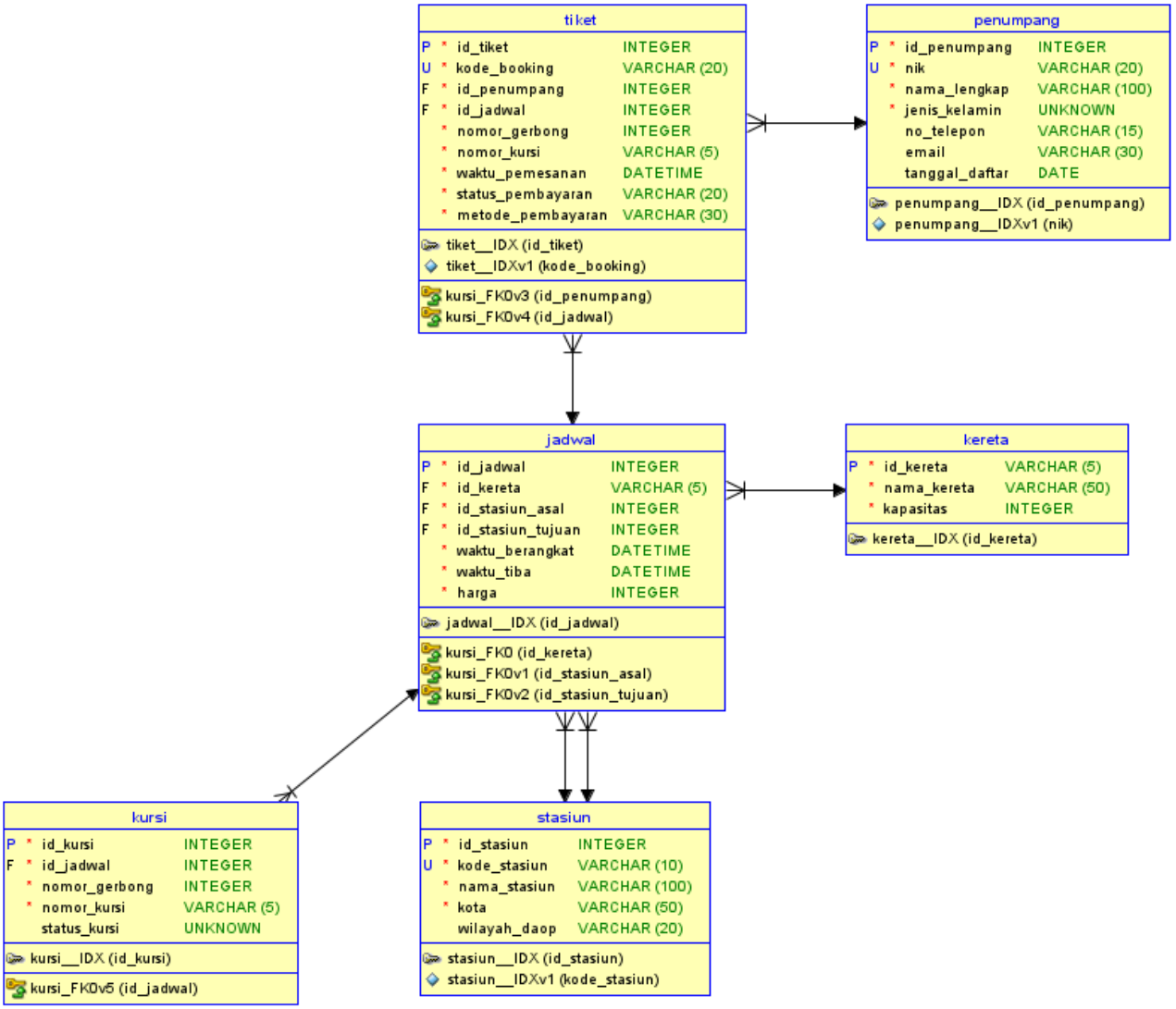
UNIVERSITAS DIPONEGORO

SEMARANG

2026

PEMBAHASAN

1. Physical Data Model



2. Index

Index digunakan untuk mempercepat proses pencarian data pada tabel terutama pada kolom yang sering digunakan dalam JOIN, WHERE, dan pencarian tertentu.

Pada database sistem pemesanan tiket kereta, beberapa kolom yang sering digunakan dalam pencarian adalah id_kereta, id_stasiun_asal, id_stasiun_tujuan, id_penumpang, id_jadwal, kode_booking, status_kursi. Oleh karena itu kolom tersebut ditambahkan index.

1. Index pada kolom id_kereta pada tabel kereta

   Index ini digunakan untuk mempercepat pencarian data jadwal berdasarkan kereta tertentu pada tabel jadwal. Tanpa adanya index pada kolom

id_kereta, database harus membaca seluruh data pada tabel untuk menemukan jadwal yang sesuai, yang dapat memperlambat proses ketika jumlah data semakin besar. Dengan menambahkan index pada kolom ini, database dapat langsung menemukan data yang dibutuhkan dengan lebih cepat. Hal ini membantu meningkatkan efisiensi query yang menggunakan kondisi WHERE maupun saat melakukan JOIN dengan tabel kereta.

- Membuat Index

```
CREATE INDEX idx_jadwal_kereta
ON jadwal(id_kereta);
```

- Mengecek Index

```
SHOW INDEX FROM jadwal;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type
jadwal	0	PRIMARY	1	id_jadwal	A	100	BTREE
jadwal	1	idx_jadwal_kereta	1	id_kereta	A	100	BTREE
jadwal	1	idx_jadwal_stasiun_asal	1	id_stasiun_asal	A	50	BTREE
jadwal	1	idx_jadwal_stasiun_tujuan	1	id_stasiun_tujuan	A	50	BTREE

2. Index pada kolom id_stasiun_asal pada tabel jadwal

Index ini digunakan untuk mempercepat pencarian data jadwal kereta berdasarkan stasiun keberangkatan pada tabel jadwal. Tanpa adanya index pada kolom id_stasiun_asal, database harus memeriksa seluruh data pada tabel untuk menemukan jadwal yang berangkat dari stasiun tertentu, yang dapat memperlambat proses jika jumlah data banyak. Dengan menambahkan index pada kolom ini, database dapat menemukan data yang sesuai dengan lebih cepat sehingga meningkatkan efisiensi query yang menggunakan kondisi WHERE untuk mencari jadwal berdasarkan stasiun keberangkatan.

- Membuat Index

```
CREATE INDEX idx_jadwal_stasiun_asal
ON jadwal(id_stasiun_asal);
```

- Mengecek Index

```
SHOW INDEX FROM jadwal;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type

jadwal	0	PRIMARY	1	id_jadwal	A	100	BTREE
jadwal	1	id_stasiun_tujuan	1	id_stasiun_tujuan	A	50	BTREE
jadwal	1	idx_jadwal_kereta	1	id_kereta	A	100	BTREE
jadwal	1	idx_jadwal_stasiun_asal	1	id_stasiun_asal	A	50	BTREE

3. Index pada kolom id_stasiun_tujuan pada tabel jadwal

   Index ini digunakan untuk mempercepat pencarian data jadwal kereta berdasarkan stasiun tujuan pada tabel jadwal. Tanpa adanya index pada kolom id_stasiun_tujuan, database harus memeriksa seluruh data dalam tabel untuk menemukan jadwal dengan tujuan tertentu, yang dapat memperlambat proses jika jumlah data cukup banyak. Dengan menambahkan index pada kolom ini, database dapat menemukan data yang dibutuhkan dengan lebih cepat sehingga meningkatkan efisiensi query yang menggunakan kondisi WHERE untuk mencari jadwal berdasarkan stasiun tujuan.

   - Membuat Index

```
CREATE INDEX idx_jadwal_stasiun_tujuan
ON jadwal(id_stasiun_tujuan);
```

   - Mengecek Index

```
SHOW INDEX FROM jadwal;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type
jadwal	0	PRIMARY	1	id_jadwal	A	100	BTREE
jadwal	1	idx_jadwal_kereta	1	id_kereta	A	100	BTREE
jadwal	1	idx_jadwal_stasiun_asal	1	id_stasiun_asal	A	50	BTREE
jadwal	1	idx_jadwal_stasiun_tujuan	1	id_stasiun_tujuan	A	50	BTREE

4. Index pada kolom id_penumpang pada tabel tiket

   Index ini digunakan untuk mempercepat pencarian data tiket yang dimiliki oleh penumpang tertentu pada tabel penumpang. Tanpa adanya index pada kolom id_penumpang, database harus memeriksa seluruh data dalam tabel tiket setipa seorang pengguna ingin melihat riwayat perjalanannya, yang mana akan memperlambat perfoma sistem seiring bertambahnya jumlah

transaksi. Dengan menambahkan index pada kolom ini, database dapat menemukan data yang dibutuhkan dengan lebih cepat dan efisien.

- Membuat Index

```
CREATE INDEX idx_tiket_id_penumpang
ON tiket(id_penumpang);
```

- Mengecek Index

```
SHOW INDEX FROM tiket;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type
tiket	0	PRIMARY	1	id_tiket	A	1	BTREE
tiket	0	kode_booking	1	kode_booking	A	1	BTREE
tiket	1	id_jadwal	1	id_jadwal	A	1	BTREE
tiket	1	idx_tiket_id_penumpang	1	id_penumpang	A	100	BTREE

5. Index pada kolom id_jadwal pada tabel tiket

Index ini digunakan untuk mempercepat proses penggabungan (JOIN) antara tabel tiket dan tabel jadwal. Tanpa adanya index pada kolom id_jadwal, sistem akan membutuhkan waktu yang lama saat harus mengidentifikasi penumpang mana saja yang terdaftar pada jadwal keberangkatan tertentu. Dengan adanya index ini, database dapat melakukan pencarian relasi antar tabel dengan lebih cepat, sehingga query yang digunakan untuk melihat daftar penumpang per jadwal atau menghitung sisa kursi yang tersedia menjadi lebih ringan dan cepat.

- Membuat Index

```
CREATE INDEX idx_tiket_id_jadwal
ON tiket(id_jadwal);
```

- Mengecek Index

```
SHOW INDEX FROM tiket;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type
tiket	0	PRIMARY	1	id_tiket	A	1	BTREE
tiket	0	kode_booking	1	kode_booking	A	1	BTREE
tiket	1	idx_tiket_id_penump	1	id_penumpang	A	100	BTREE

		ang					
tiket	1	idx_tiket_id_jadwal	1	id_jadwal	A	93	BTREE

6. Index pada kolom Id_Jadwal dan Status_Kursi pada tabel Kursi

Index ini digunakan untuk mempercepat pencarian ketersediaan kursi secara spesifik berdasarkan jadwal keberangkatan dan status hunian kursi. Tanpa adanya index pada id_jadwal dan status_kursi, proses krusial seperti pemilihan kursi oleh penumpang atau pemantauan sisa kursi yang tersedia pada jam tertentu akan memakan waktu lama karena database harus memindai seluruh data satu per satu. Indeks gabungan ini memungkinkan sistem untuk langsung menyaring data kursi yang relevan dengan jadwal tertentu, sehingga menjamin kecepatan operasional aplikasi saat *high traffic* dan keakuratan tampilan denah kursi bagi pengguna secara *real-time*.

- Membuat Index

```
CREATE INDEX idx_jadwal_status ON
kursi(id_jadwal, status_kursi);
```

- Mengecek Index

```
SHOW INDEX FROM kursi;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type
tiket	0	PRIMARY	1	id_tiket	A	1	BTREE
tiket	0	kode_booking	1	kode_booking	A	1	BTREE
tiket	1	idx_tiket_id_penumpang	1	id_penumpang	A	100	BTREE
tiket	1	idx_tiket_id_jadwal	1	id_jadwal	A	93	BTREE

3. User Role

Pada bagian ini, dilakukan penerapan Role-Based Access Control (RBAC) untuk memastikan setiap pengguna hanya memiliki hak akses yang sesuai dengan fungsinya (Principle of Least Privilege). Setiap *user* hanya diberikan hak akses yang benar-benar dibutuhkan untuk menyelesaikan tugasnya. Hal ini bertujuan untuk meminimalkan risiko 'human error' (seperti ketidaksengajaan menghapus tabel) atau penyalahgunaan akun (internal threat).

1. Definisi Peran (Roles)

   Terdapat tiga peran utama yang dirancang untuk mengelola sistem tiketing kereta api ini:

   - admin_role (Superuser): Peran ini memegang kendali penuh atas seluruh aspek basis data, mencakup kewenangan DDL (Data Definition Language) untuk mengubah struktur tabel, DML (Data Manipulation Language) untuk mengelola seluruh data, serta DCL (Data Control Language) untuk mengatur hak akses pengguna lain. Administrator bertanggung jawab penuh atas pemeliharaan infrastruktur dan keamanan sistem secara menyeluruh.
   - staff_role (Operational): Dirancang untuk staf di garda depan (petugas tiket dan administrasi), peran ini berfokus pada operasional harian. Staf diberikan izin untuk melakukan manipulasi data (Insert/Update) pada tabel-tabel transaksional seperti penumpang, tiket, dan kursi. Namun, untuk menjaga integritas sistem, akses terhadap data master seperti jadwal, kereta, dan stasiun dibatasi hanya untuk dibaca (Read-only). Selain itu, fitur penghapusan data bersifat terbatas dan hanya dapat dilakukan melalui Stored Procedure yang telah divalidasi oleh sistem.
   - manager_role (Auditor): Peran ini ditujukan untuk pihak manajemen yang memerlukan visibilitas data guna pengambilan keputusan. Sesuai fungsinya, manager_role diberikan hak akses Read-only di seluruh skema database. Hal ini memungkinkan manajer untuk memantau okupansi kursi, tren penjualan tiket, dan melakukan Evaluasi Kueri (analisis performa) tanpa risiko terjadinya perubahan data operasional yang tidak disengaja.

2. Implementasi RBAC (SQL Scripts)

   Kueri CREATE ROLE digunakan untuk mendefinisikan grup otorisasi. Penggunaan role lebih efisien dibandingkan memberikan hak akses ke setiap user secara manual, karena mempermudah manajemen hak akses jika di masa depan terdapat penambahan staf atau perubahan kebijakan keamanan. Berikut adalah perintah SQL yang digunakan untuk membentuk struktur keamanan tersebut:

```
-- 1. Membuat Role
CREATE ROLE 'admin_role', 'staff_role', 'manager_role';

-- 2. Memberikan Hak Akses ke admin_role (Full Access)
GRANT ALL PRIVILEGES ON *.* TO 'admin_role';

-- 3. Memberikan Hak Akses ke staff_role (Operational)
```

```
GRANT SELECT, INSERT, UPDATE ON penumpang TO
'staff_role';
GRANT SELECT, INSERT, UPDATE ON tiket TO 'staff_role';
GRANT SELECT, INSERT, UPDATE ON kursi TO 'staff_role';
GRANT SELECT ON kereta TO 'staff_role';
GRANT SELECT ON stasiun TO 'staff_role';
GRANT SELECT ON jadwal TO 'staff_role';

-- Akses Stored Procedure untuk Staff
GRANT EXECUTE ON PROCEDURE sp_TiketPerKode TO
'staff_role';
GRANT EXECUTE ON PROCEDURE sp_PenumpangPerHari TO
'staff_role';
GRANT EXECUTE ON PROCEDURE sp_delete_penumpang TO
'staff_role';

-- 4. Memberikan Hak Akses ke manager_role (Monitoring)
GRANT SELECT ON * TO 'manager_role';
GRANT EXECUTE ON PROCEDURE sp_TiketPerKode TO
'manager_role';
GRANT EXECUTE ON PROCEDURE sp_PenumpangPerHari TO
'manager_role';
```

Meskipun staf operasional memiliki wewenang untuk menghapus data penumpang yang melakukan pembatalan, mereka tidak diberikan hak akses DELETE secara langsung pada tabel penumpang atau tiket. Sebagai gantinya, staf hanya diberikan izin EXECUTE pada prosedur sp_delete_penumpang. Hal ini memastikan bahwa setiap penghapusan data selalu mengikuti logika bisnis yang benar (menghapus data di tabel anak terlebih dahulu sebelum tabel induk) untuk menjaga Integritas Referensial database. Berikut adalah tampilan perizinan yang dimiliki oleh setiap role untuk mengakses tabel pada database.

```
MariaDB [kuliah_mbd_prauts]> SHOW GRANTS FOR
'staff_role';
+-------------------------------------------------------
-------------------------------+
| Grants for staff_role
|
+-------------------------------------------------------
-------------------------------+
| GRANT USAGE ON *.* TO `staff_role`
|
| GRANT SELECT ON `kuliah_mbd_prauts`.`stasiun` TO
`staff_role`                          |
| GRANT SELECT ON `kuliah_mbd_prauts`.`jadwal` TO
`staff_role`                           |
| GRANT SELECT ON `kuliah_mbd_prauts`.`kereta` TO
```

```
`staff_role`                          |
| GRANT SELECT, INSERT, UPDATE ON
`kuliah_mbd_prauts`.`kursi` TO `staff_role`          |
| GRANT SELECT, INSERT, UPDATE ON
`kuliah_mbd_prauts`.`tiket` TO `staff_role`          |
| GRANT SELECT, INSERT, UPDATE ON
`kuliah_mbd_prauts`.`penumpang` TO `staff_role`      |
| GRANT EXECUTE ON PROCEDURE
`kuliah_mbd_prauts`.`sp_tiketperkode` TO `staff_role`
|
| GRANT EXECUTE ON PROCEDURE
`kuliah_mbd_prauts`.`sp_delete_penumpang` TO
`staff_role` |
+-------------------------------------------------------
-------------------------------+
9 rows in set (0.000 sec)
```

```
MariaDB [kuliah_mbd_prauts]> SHOW GRANTS FOR
'admin_role';
+-------------------------------------------------------
--------+
| Grants for admin_role
|
+-------------------------------------------------------
--------+
| GRANT ALL PRIVILEGES ON *.* TO `admin_role`
|
| GRANT ALL PRIVILEGES ON `kuliah_mbd_prauts`.* TO
`admin_role` |
+-------------------------------------------------------
--------+
2 rows in set (0.000 sec)
```

```
MariaDB [kuliah_mbd_prauts]> SHOW GRANTS FOR
'manager_role';
+-------------------------------------------------------
-----------------------------+
| Grants for manager_role
|
+-------------------------------------------------------
-----------------------------+
| GRANT USAGE ON *.* TO `manager_role`
|
| GRANT SELECT ON `kuliah_mbd_prauts`.* TO
`manager_role`                              |
| GRANT EXECUTE ON PROCEDURE
`kuliah_mbd_prauts`.`sp_tiketperkode` TO `manager_role`
|
```

```
+-------------------------------------------------------
-----------------------------+
3 rows in set (0.000 sec)
```

3. Manajemen Pengguna

```
-- Membuat User Baru
CREATE USER 'admin1'@'localhost' IDENTIFIED BY
'PassAdmin123!';
CREATE USER 'staff1'@'localhost' IDENTIFIED BY
'PassStaff123!';
CREATE USER 'manager1'@'localhost' IDENTIFIED BY
'PassManager123!';

-- Menugaskan Role ke User
GRANT 'admin_role' TO 'admin1'@'localhost';
GRANT 'staff_role' TO 'staff1'@'localhost';
GRANT 'manager_role' TO 'manager1'@'localhost';

-- Mengaktifkan Role Default
SET DEFAULT ROLE 'admin_role' FOR 'admin1'@'localhost';
SET DEFAULT ROLE 'staff_role' FOR 'staff1'@'localhost';
SET DEFAULT ROLE 'manager_role' FOR
'manager1'@'localhost';
```

Dengan skema otoritas ini, database sistem tiketing kereta api telah memenuhi standar keamanan dasar yang memisahkan antara administrator sistem, operator data, dan pengambil keputusan (manager). Langkah ini menjamin ketersediaan data (*availability*) sekaligus kerahasiaan (*confidentiality*) informasi sensitif penumpang.